МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

«ВЛАДИВОСТОКСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВВГУ»)

Институт информационных технологий и анализа данных
Кафедра информационных технологий и систем

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

по дисциплине «Информатика и основы программирования»

Лабораторная работа 6. Реализация численных алгоритмов

Студент
гр. БИН-26-03 _____ А. В. Лямин

Студент
гр. БИН-26-03 _____ А. В. Тимченко

Преподаватель _____ М. В. Водяницкий

Владивосток 2026

1 Выполнение работы

1.1 Задание 1. Наибольший общий делитель

В первом задании требуется реализовать алгоритм Евклида для нахождения наибольшего общего делителя (НОД) двух целых чисел. Пользователь вводит целые числа a и b на отдельных строках. Если хотя бы одно число отрицательное или оба числа равны нулю, программа выводит «Ошибка». Для остальных значений вычисляется НОД: пара чисел последовательно заменяется по правилу $(a, b) = (b, a \bmod b)$ до тех пор, пока b не станет нулем. Первое число полученной пары является искомым результатом.

Алгоритм решения первого задания представлен на рисунке 1.

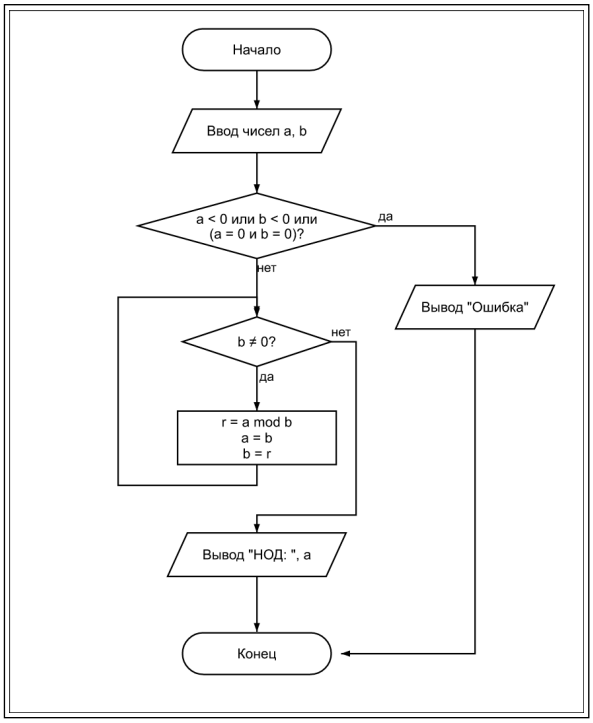


Рисунок 1 – Блок-схема программы для задания 1

На блок-схеме показаны ввод исходных чисел, проверка ограничений, цикл с предусловием $b \neq 0$ для вычисления остатка от деления и последовательного переприсваивания значений, а также вывод найденного значения НОД. Блок-схема оформлена по ГОСТ 19.701-90.

Текст программы для задания 1 представлен на рисунке 2.

```
a = int(input())
b = int(input())
if a < 0 or b < 0 or (a == 0 and b == 0):
    print('Ошибка')
else:
    while b != 0:
        a, b = b, a % b
    print(f'НОД: {a}')
```

Рисунок 2 – Текст программы для задания 1

В программе используется множественное присваивание `a, b = b, a % b` внутри цикла `while`, реализующее шаг алгоритма Евклида.

Результат работы программы при вводе 84 и 30 представлен на рисунке 3.

```
84
30
НОД: 6
```

Рисунок 3 – Результат работы программы для задания 1

1.2 Задание 2. Проверка числа на простоту

Во втором задании требуется проверить, является ли введенное пользователем целое число n простым. Простое число — это целое число больше единицы, имеющее ровно два положительных делителя: единицу и само себя. Все значения меньше 2 дают результат «Не простое». Для проверки перебираются возможные делители, начиная с 2, пока квадрат делителя не превысит n. Если найден хотя бы один делитель, число признается составным.

Алгоритм решения второго задания представлен на рисунке 4.

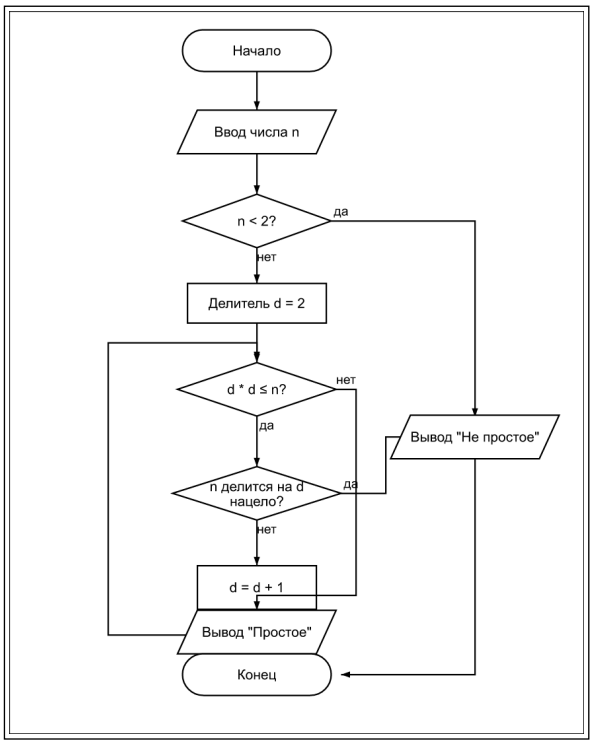


Рисунок 4 – Блок-схема программы для задания 2

На блок-схеме показаны отсечение неподходящих значений ($n < 2$), цикл перебора делителей с условием $d^2 \leq n$, проверка делимости n на делитель без остатка и вывод соответствующего вердикта. Блок-схема оформлена по ГОСТ 19.701-90.

Текст программы для задания 2 представлен на рисунке 5.

```
n = int(input())
if n < 2:
    print('Не простое')
    exit()
for i in range(2, int(n**0.5) + 1):
    if n % i == 0:
        print('Не простое')
        break
else:
    print('Простое')
```

Рисунок 5 – Текст программы для задания 2

Для ограничения границы поиска делителей используется выражение `int(n**0.5) + 1`, а блок `else` цикла `for` выполняется только в случае, если делитель не был найден.

Результат работы программы при вводе 49 представлен на рисунке 6.

```
49
Не простое
```

Рисунок 6 – Результат работы программы для задания 2

1.3 Задание 3. Разложение на простые множители

В третьем задании требуется разложить допустимое целое число n ($n \geq 2$) на простые множители и вывести их в порядке неубывания через один пробел. При $n < 2$ выводится сообщение «Ошибка». Алгоритм последовательного деления перебирает делители начиная с $d = 2$: при делении без остатка исходное число уменьшается в d раз, и проверка повторяется для того же делителя до полного разложения.

Алгоритм решения третьего задания представлен на рисунке 7.

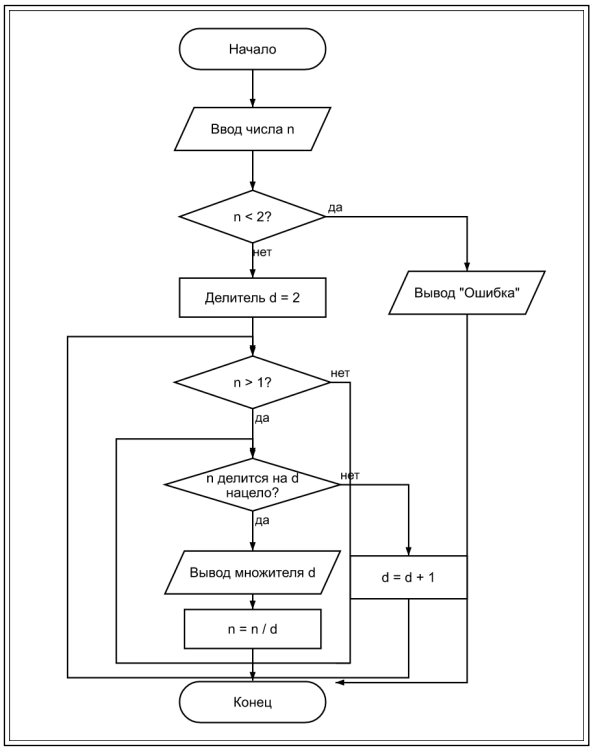


Рисунок 7 – Блок-схема программы для задания 3

На блок-схеме отражены структура из двух вложенных циклов: внешний цикл проверяет условие незавершенности факторизации ($n > 1$), а внутренний цикл последовательно выделяет кратный простой множитель d. Блок-схема оформлена по ГОСТ 19.701-90.

Текст программы для задания 3 представлен на рисунке 8.

```
n = int(input())
if n < 2:
    print('Ошибка')
    exit()
else:
    factors = []
    d = 2
    while n > 1:
        while n % d == 0:
            factors.append(d)
            n //= d
        d += 1
    print(*factors)
```

Рисунок 8 – Текст программы для задания 3

Найденные простые множители помещаются в список `factors` и выводятся на экран через операцию распаковки `print(*factors)`.

Результат работы программы при вводе 60 представлен на рисунке 9.

```
60
2 2 3 5
```

Рисунок 9 – Результат работы программы для задания 3

1.4 Задание 4. Цифры числа

В четвертом задании требуется определить количество цифр и их сумму для введенного пользователем целого числа n. Если число отрицательное, программа выводит «Ошибка». Для допустимого значения подсчитывается число знаков и сумма входящих в запись цифр.

Алгоритм решения четвертого задания представлен на рисунке 10.

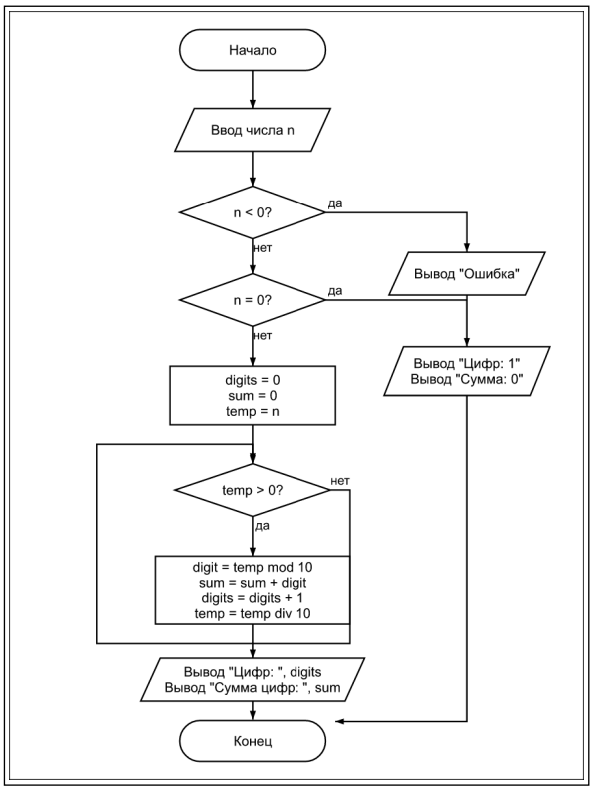


Рисунок 10 – Блок-схема программы для задания 4

На блок-схеме показаны проверка допустимости знака числа, цикл последовательного извлечения цифр и накопления счетчика с суммой, а также итоговый вывод результатов. Блок-схема оформлена по ГОСТ 19.701-90.

Текст программы для задания 4 представлен на рисунке 11.

```
n = int(input())
if n < 0:
    print('Ошибка')
    exit()
digits = 0
sum = 0
for digit in str(n):
    digits += 1
    sum += int(digit)
print(f'Цифр: {digits}')
print(f'Сумма цифр: {sum}')
```

Рисунок 11 – Текст программы для задания 4

Программа выполняет итерацию по строковому представлению числа, подсчитывая количество символов и суммируя их числовые значения `int(digit)`.

Результат работы программы при вводе 4050 представлен на рисунке 12.

```
4050
Цифр: 4
Сумма цифр: 9
```

Рисунок 12 – Результат работы программы для задания 4

1.5 Задание 5. Таблица расстояний

В пятом задании требуется построить таблицу расстояний между точками с целочисленными координатами от 0 до $n - 1$, расположенными на прямой. Пользователь вводит целое число n от 1 до 10 включительно. При выходе за диапазон выводится «Ошибка». В строке i и столбце j выводится расстояние между точками i и j, равное большей координате минус меньшая. Элементы строки разделяются одним пробелом без пробела в конце.

Алгоритм решения пятого задания представлен на рисунке 13.

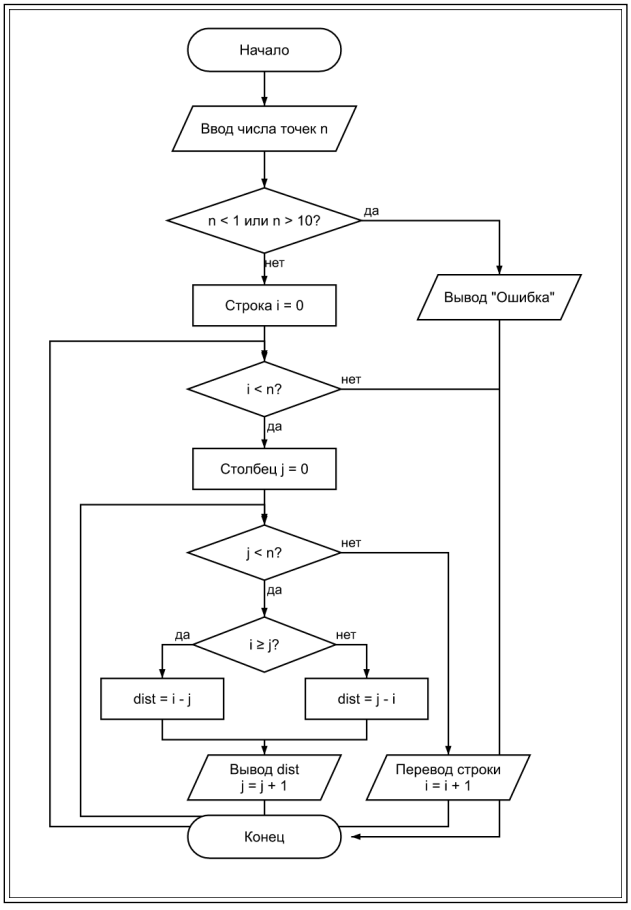


Рисунок 13 – Блок-схема программы для задания 5

На блок-схеме отражены проверка допустимости n, организация двух вложенных циклов для перебора строк i и столбцов j, условный выбор порядка вычитания координат i и j для вычисления модуля разности и поэлементный вывод строки таблицы. Блок-схема оформлена по ГОСТ 19.701-90.

Текст программы для задания 5 представлен на рисунке 14.

```
n = int(input())
if n not in range(1, 11):
    print('Ошибка')
    exit()
for i in range(n):
    row = []
    for j in range(n):
        if i > j:
            dist = i - j
        else:
            dist = j - i
        row.append(str(dist))
    print(' '.join(row))
```

Рисунок 14 – Текст программы для задания 5

Для каждой строки формируется список строковых значений `row`, который затем соединяется пробелами через `' '.join(row)` и выводится на экран.

Результат работы программы при вводе 4 представлен на рисунке 15.

```
4
0 1 2 3
1 0 1 2
2 1 0 1
3 2 1 0
```

Рисунок 15 – Результат работы программы для задания 5